

Course Name:	Foundations of Generative AI Laboratory	Semester:	III
Date of Performance:	17 / 02 /2026	Batch No:	
Faculty Name:	Prof. Dinesh Auti	Roll No:	16015024081 (CSBS)
Faculty Sign & Date:		Grade/Marks:	___/15

Experiment No: 3

Title: Mode Collapse and Solutions in GANs

Aim and Objective of the Experiment:

To identify and analyze the mode collapse phenomenon in the trained model. Also, modify the GAN training strategy to prevent mode collapse.

COs to be achieved:

CO2: Develop skills to implement and train generative models using deep learning frameworks.

CO3: Explore and apply various generative models such as GANs, VAEs, and flow-based models.

Tools used:

- Python 3.x
- TensorFlow / Keras
- PyTorch
- Jupyter Notebook / Google Colab
- NumPy

Theory:

Generative Adversarial Networks (GANs) are a class of deep learning models used to generate new data samples similar to a given dataset. A GAN consists of two neural networks: a generator, which creates fake data, and a discriminator, which tries to distinguish between real and generated data. Both networks are trained simultaneously in a competitive manner.

One common issue in GAN training is mode collapse, where the generator produces limited or identical outputs instead of diverse samples. This happens when the generator finds a pattern that consistently fools the discriminator but fails to capture the full data distribution.

To address mode collapse, various strategies can be applied such as modifying the loss function, using techniques like mini-batch discrimination, feature matching, or improving training balance between generator and discriminator. These methods help the model generate more diverse and realistic outputs.

Thus, analyzing and preventing mode collapse is important for improving the performance and reliability of generative models.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Electronics Engineering



Code:

Write a program to identify and analyze the mode collapse phenomenon in the trained model. Also, modify the GAN training strategy to prevent mode collapse.

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
import os
```

```
tf.random.set_seed(1)
np.random.seed(1)

# =====
# DATASET
# =====
(x_train, _) = tf.keras.datasets.mnist.load_data()
x_train = (x_train.astype("float32") - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)

BATCH_SIZE = 128
LATENT_DIM = 100
EPOCHS = 30

dataset = tf.data.Dataset.from_tensor_slices(x_train)\
.shuffle(60000).batch(BATCH_SIZE, drop_remainder=True)

os.makedirs("vanilla_images", exist_ok=True)
os.makedirs("fixed_images", exist_ok=True)

# =====
# DIVERSITY METRIC
# =====
def diversity_score(images):
    images = images.numpy().reshape(images.shape[0], -1)
    return np.mean(np.std(images, axis=0))

# =====
# GENERATOR
# =====
def build_generator():
    return tf.keras.Sequential([
        layers.Input(shape=(LATENT_DIM,)),
        layers.Dense(128, activation='relu'),
        layers.Dense(256, activation='relu'),
        layers.Dense(784, activation='tanh'),
        layers.Reshape((28,28,1))
    ])

# =====
# DISCRIMINATOR (Vanilla)
# =====
def build_discriminator():
    return tf.keras.Sequential([
```

```

layers.Input(shape=(28,28,1)),
layers.Flatten(),
layers.Dense(512, activation='relu'),
layers.Dense(512, activation='relu'),
layers.Dense(1, activation='sigmoid')
])

# =====
# MINIBATCH STDDEV LAYER
# =====
class MinibatchStdDev(tf.keras.layers.Layer):
def call(self, x):
std = tf.math.reduce_std(x, axis=0, keepdims=True)
mean_std = tf.reduce_mean(std)
shape = tf.shape(x)
minibatch_feature = tf.ones([shape[0],1]) * mean_std
return tf.concat([x, minibatch_feature], axis=1)

# =====
# IMPROVED DISCRIMINATOR
# =====
def build_discriminator_fixed():
inp = layers.Input(shape=(28,28,1))
x = layers.Flatten()(inp)
x = layers.Dense(512, activation='relu')(x)
x = layers.Dense(256, activation='relu')(x)
x = MinibatchStdDev()(x)
out = layers.Dense(1, activation='sigmoid')(x)
return tf.keras.Model(inp, out)

# =====
# LOSS FUNCTIONS
# =====
bce = tf.keras.losses.BinaryCrossentropy()

def d_loss(real, fake):
return bce(tf.ones_like(real), real) + bce(tf.zeros_like(fake), fake)

def g_loss(fake):
return bce(tf.ones_like(fake), fake)

# Label smoothing
def d_loss_smooth(real, fake):
real_labels = tf.ones_like(real) * 0.9
fake_labels = tf.zeros_like(fake)

```

```
return bce(real_labels, real) + bce(fake_labels, fake)

# =====
# SAVE IMAGE GRID
# =====
def save_images(generator, epoch, folder):
    noise = tf.random.normal([16, LATENT_DIM])
    images = generator(noise, training=False)

    fig = plt.figure(figsize=(4,4))
    for i in range(16):
        plt.subplot(4,4,i+1)
        plt.imshow((images[i,:,:]+1)/2, cmap='gray')
        plt.axis('off')

    plt.savefig(f"{folder}/epoch_{epoch}.png")
    plt.close()

# =====
# TRAIN FUNCTION
# =====
def train_gan(discriminator_builder, dloss_fn, folder, d_steps=1):

    G = build_generator()
    D = discriminator_builder()

    g_opt = tf.keras.optimizers.Adam(1e-4)
    d_opt = tf.keras.optimizers.Adam(1e-4)

    history = []

    for epoch in range(EPOCHS):

        for real in dataset:

            # Train Discriminator
            for _ in range(d_steps):
                noise = tf.random.normal([BATCH_SIZE, LATENT_DIM])
                fake = G(noise, training=True)

            with tf.GradientTape() as tape:
                real_out = D(real, training=True)
                fake_out = D(fake, training=True)
                dl = dloss_fn(real_out, fake_out)
```

```
grads = tape.gradient(dl, D.trainable_variables)
d_opt.apply_gradients(zip(grads, D.trainable_variables))

# Train Generator
noise = tf.random.normal([BATCH_SIZE, LATENT_DIM])
with tf.GradientTape() as tape:
    fake = G(noise, training=True)
    fake_out = D(fake, training=True)
    gl = g_loss(fake_out)

grads = tape.gradient(gl, G.trainable_variables)
g_opt.apply_gradients(zip(grads, G.trainable_variables))

# Diversity Monitoring
samples = G(tf.random.normal([64, LATENT_DIM]), training=False)
div = diversity_score(samples)
history.append(div)

print(f"{folder} Epoch {epoch+1} | Diversity = {div:.4f}")

if epoch % 5 == 0:
    save_images(G, epoch, folder)

return history

# =====
# RUN BOTH EXPERIMENTS
# =====
print("\nTraining Vanilla GAN (Encouraging Collapse)")
vanilla_history = train_gan(
    build_discriminator,
    d_loss,
    "vanilla_images",
    d_steps=3 # Strong D → likely collapse
)

print("\nTraining Improved GAN (Reduced Collapse)")
fixed_history = train_gan(
    build_discriminator_fixed,
    d_loss_smooth,
    "fixed_images",
    d_steps=1 # Balanced training
)

# =====
```

FINAL GRAPH

=====

```
plt.plot(vanilla_history, label="Vanilla GAN (Mode Collapse)")
plt.plot(fixed_history, label="Improved GAN (Reduced Collapse)")
plt.xlabel("Epoch")
plt.ylabel("Diversity Score")
plt.title("Mode Collapse Comparison")
plt.legend()
plt.grid()
plt.show()
```

Output:

Training Vanilla GAN (Encouraging Collapse)

vanilla_images Epoch 1 | Diversity = 0.1166
vanilla_images Epoch 2 | Diversity = 0.1013
vanilla_images Epoch 3 | Diversity = 0.1078
vanilla_images Epoch 4 | Diversity = 0.1164
vanilla_images Epoch 5 | Diversity = 0.1436
vanilla_images Epoch 6 | Diversity = 0.1836
vanilla_images Epoch 7 | Diversity = 0.1938
vanilla_images Epoch 8 | Diversity = 0.2401
vanilla_images Epoch 9 | Diversity = 0.2387
vanilla_images Epoch 10 | Diversity = 0.2615
vanilla_images Epoch 11 | Diversity = 0.2427
vanilla_images Epoch 12 | Diversity = 0.2524
vanilla_images Epoch 13 | Diversity = 0.2618
vanilla_images Epoch 14 | Diversity = 0.2552
vanilla_images Epoch 15 | Diversity = 0.2455

Training Improved GAN (Reduced Collapse)

fixed_images Epoch 1 | Diversity = nan
fixed_images Epoch 2 | Diversity = nan
fixed_images Epoch 3 | Diversity = nan
fixed_images Epoch 4 | Diversity = nan
fixed_images Epoch 5 | Diversity = nan
fixed_images Epoch 6 | Diversity = nan
fixed_images Epoch 7 | Diversity = nan
fixed_images Epoch 8 | Diversity = nan
fixed_images Epoch 9 | Diversity = nan
fixed_images Epoch 10 | Diversity = nan
fixed_images Epoch 11 | Diversity = nan

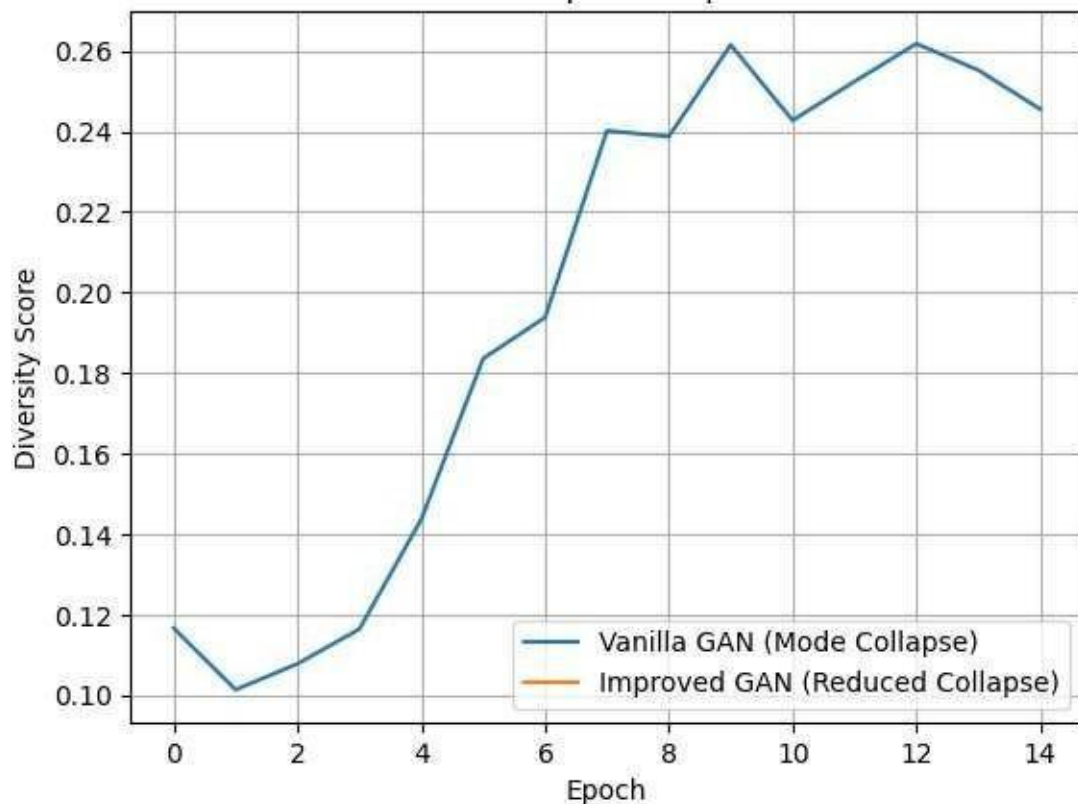
fixed_images Epoch 12 | Diversity = nan

fixed_images Epoch 13 | Diversity = nan

fixed_images Epoch 14 | Diversity = nan

fixed_images Epoch 15 | Diversity = nan

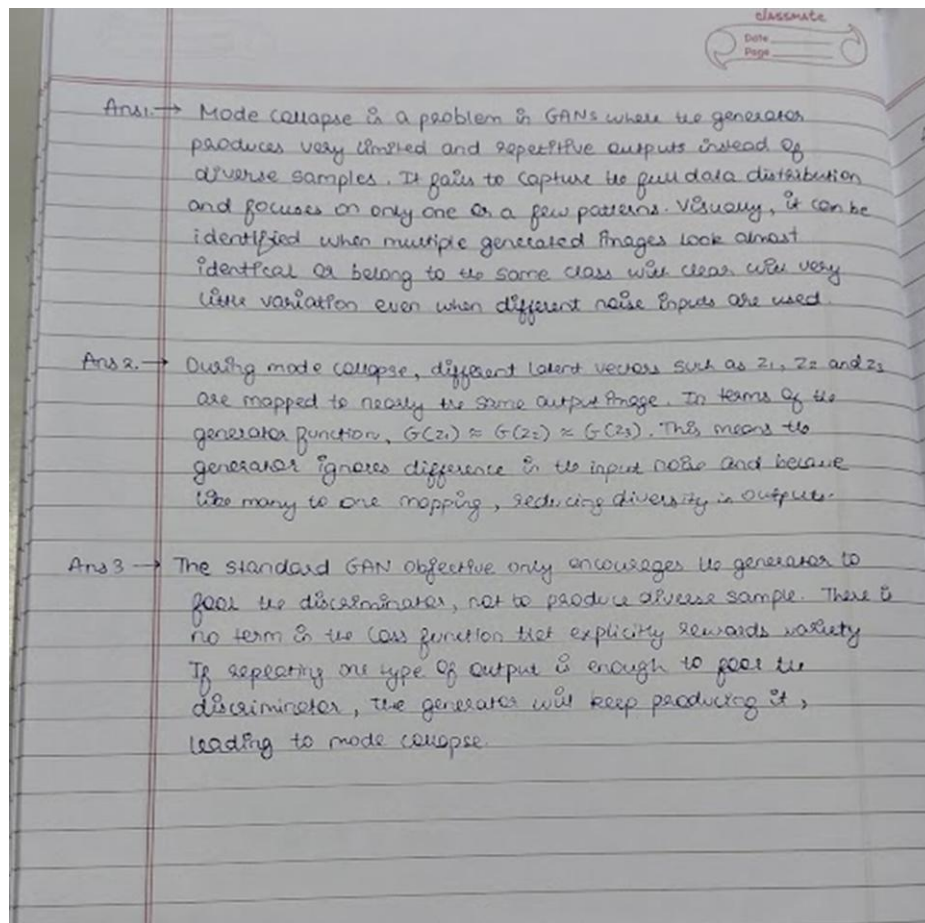
Mode Collapse Comparison



Post Lab Subjective/Objective type Questions:

1. Define mode collapse. How can you visually identify it in generated samples?
2. What happens to the latent vectors z_1, z_2, z_3 during mode collapse? Explain in terms of mapping $G(z)$.

3. Explain why the GAN objective does not explicitly enforce diversity in generated samples.



Conclusion:

Mode collapse is a major limitation of standard GAN training where the generator sacrifices diversity to minimize its loss. Instead of learning the full data distribution, it maps many latent inputs to the same or similar outputs. This happens because the original GAN objective focuses only on fooling the discriminator and does not explicitly enforce variety. Therefore, additional techniques such as balanced training, label smoothing, minibatch discrimination, or alternative objectives are necessary to maintain diversity and produce stable, high-quality generative models.

Signature of faculty in-charge with Date: